CSA0919 – Programming in Java
Assignment 1 – Campus Smart-Parking System

TASK 5 – Apply static Members and the Object Life Cycle

Field	Details
Student Name	Juhi Surin
Register Number	192525148
Course	CSA0919 – Programming in Java
Task	Task 5 (3 Marks) – Bloom's Level K4 – Analyse
Java used for execution check	Java 21 (OpenJDK 21.0.12). Source uses only syntax valid in Java 21 and later, including Java 27.
GitHub repository	CSA09-192525148-Assignment1 / Task05 (add link before submission)

1. Objective

The objective of Task 5 is to identify the data and behaviour that belong to a **class as a whole** (static members) and the data that belong to **individual objects** (instance members) in the Campus Smart-Parking System. The task also traces the life cycle of a `ParkingTicket` object from creation to garbage-collection eligibility and analyses how objects are passed to and returned from methods such as `ParkingService.enter(...)` and `exit(...)`.

Consistency note: The classes `Vehicle`, `ParkingTicket`, `ParkingSession`, `ExitReceipt`, `ParkingFacility` and `ParkingService` follow the Task 1–4 design. `ParkingFacility.allocate()/release()` are the Task 3 methods unchanged in logic; only the facility name, the instance field `occupancy` and the static field `totalOccupancy` (with their getters) were added. Vehicle sub-classes (Task 6 onwards) are deliberately not introduced; `Vehicle` carries a simple type string here.

2. Static Members in the Campus Smart-Parking System

Static versus instance members

An **instance member** belongs to one object; every object gets its own copy and it is reached through an object reference. A **static member** belongs to the class; exactly one copy exists per class (per class loader), it is created when the class is initialised, and it can be reached with the class name without creating any object.

A. Static Variables

Global ticket counter. The ticket sequence is data about *all* tickets, not about one ticket. It is therefore declared in `ParkingTicket` as:

```
private static final AtomicLong SEQUENCE = new AtomicLong(0);
```

- **Why it belongs to the class:** if each ticket owned a counter, every new ticket would start from its own zero and the counter could never produce a different number for the next ticket. The sequence must outlive any single ticket and be common to all of them.
- **Why it is shared across North Gate and South Gate:** the university needs ticket numbers that never repeat anywhere in the running application (Task 1 rule: a ticket number is never reused). One class-level counter serves both facilities, so no two tickets, in either facility, receive the same number.
- **How AtomicLong supports safe concurrent generation:** `incrementAndGet()` performs the read-increment-write as a single atomic operation. If entry gates in two facilities (two threads) issue tickets simultaneously, each call receives a distinct value. A plain `long` with `++` is not atomic and could hand the same number to two tickets.
- **Why each ticket still stores its own number:** the static counter only *generates* numbers. The generated value is copied into the instance field `private final long ticketNo`, so a ticket keeps the identity it was issued with, while the counter moves on.

Static constants. Fixed configuration shared by all vehicles, the hourly rate table, is a `private static final Map` in `Vehicle` (filled in a static block, see C). Constants never change per object, so one copy is enough.

Occupancy: system-wide total (static) versus per-facility (instance)

`ParkingFacility` keeps **two** different occupancy counters, because the system needs two different facts:

```
private int occupancy;                // instance: vehicles parked in THIS facility
private static int totalOccupancy;    // static:   vehicles parked across ALL facilities
```

- **Why `totalOccupancy` is static:** the total across every facility is a fact about the class as a whole, not about North Gate or South Gate. One shared variable is incremented by *every* ParkingFacility object whenever any vehicle is successfully allocated and decremented whenever any slot is successfully released, so it always equals the sum of all facilities. It is read with `ParkingFacility.getTotalOccupancy()`, a static getter that needs no facility object.
- **Why `occupancy` is an instance variable:** North Gate and South Gate are independent facilities with their own slots, so each needs its own count. If this value were static, parking a car at North Gate would also raise South Gate's figure and no facility could report its own occupancy. Each object therefore keeps one private copy, read through `getOccupancy()`.
- **Both counters change together and only on success:** `allocate()` increments both after a free slot is found; `release()` decrements both only after a valid, occupied slot is freed. A full facility (allocate returns null) or an invalid or repeated release leaves both counters unchanged.
- **Demonstrated result:** after two vehicles enter North Gate and one enters South Gate the program prints *North occupancy = 2, South occupancy = 1, Total occupancy = 3*; after one North Gate vehicle exits it prints *North occupancy = 1, South occupancy = 1, Total occupancy = 2*.

B. Static Methods

A static method belongs to the class and does not receive a `this` reference. It suits operations that depend only on their parameters. In this system:

- `Vehicle.normalizeRegNo(String)` – removes spaces and hyphens and upper-cases the registration number, so “ tn-01 ab 1111 ” becomes TN01AB1111. It needs no Vehicle object.
- `Vehicle.billableHours(long minutes)` – integer ceiling `(minutes + 59) / 60`; independent of any vehicle.
- `Vehicle.rateFor(String type)` – looks up the static rate table.

```
public static String normalizeRegNo(String raw) {
    if (raw == null || raw.isBlank()) throw new IllegalArgumentException("Registration number is required");
    return raw.replaceAll("[\\s-]", "").toUpperCase();
}
public static long billableHours(long minutes) { return (minutes + 59) / 60; }
```

Why a static method cannot directly use instance variables: instance variables exist only inside an object, and a static method is invoked without any object, so the compiler cannot know *whose* `regNo` is meant. Writing `regNo` inside `normalizeRegNo` would be a compile-time error (“non-static variable cannot be referenced from a static context”). A static method can use instance data only through an explicit reference, e.g. `vehicle.getRegNo()`.

C. Static Initialization Block

A static initialization block (`static { ... }`) is code that runs **once, when the class is initialised** by the JVM (at the first creation of an object, first static-method call or first static-field access, whichever comes first), before any object of the class exists. It is suitable for one-time class-level setup that needs more than a single expression, such as filling a lookup table or validating configuration.

```
private static final Map<String, Integer> RATE_PER_HOUR;
static {
    System.out.println("[Vehicle static block] executed once at class initialisation");
    RATE_PER_HOUR = Map.of("TWO_WHEELER", 10, "CAR", 30, "EV", 30);
```

```
}
```

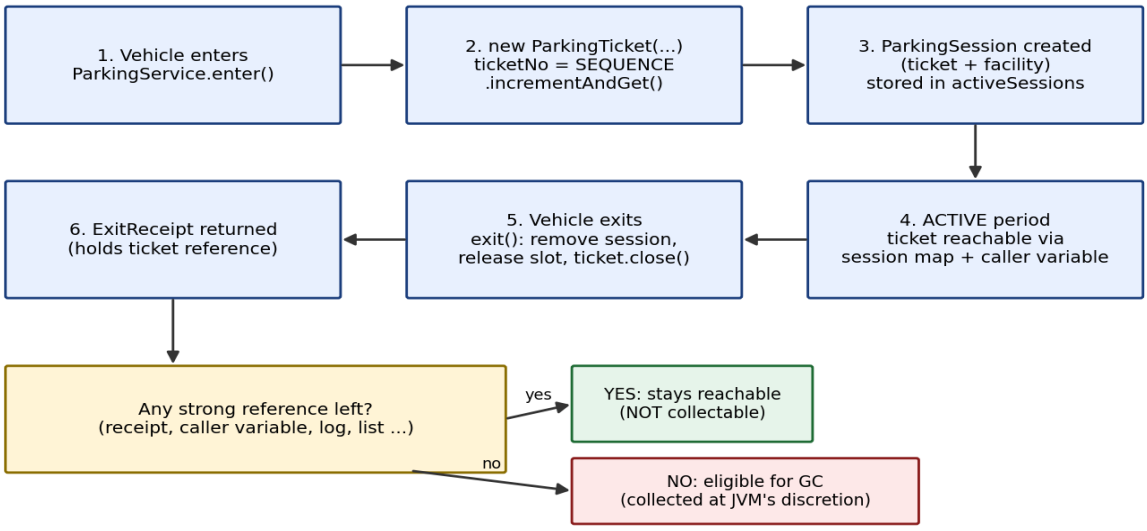
In the demonstration the message appears exactly once, at the first use of `Vehicle`, even though several Vehicle objects are created later. A static block is **not necessary** for the ticket counter: `new AtomicLong(0)` is a single expression, so direct field initialisation is sufficient and simpler.

Comparison of the four member kinds

Aspect	Static variable	Instance variable	Static method	Static init block
Belongs to	Class	Each object	Class	Class
Copies	One	One per object	One (no per-object state)	Runs once
Created / runs	At class initialisation	At object construction	When called	At class initialisation, before first use
Accessed by	ClassName.field	objectRef.field	ClassName.method()	Not called; executed by JVM
Can use instance data directly?	–	–	No (needs object reference)	No
Parking example	ParkingTicket.SEQUENCE; ParkingFacility.totalOccupancy; rate table	regNo, ticketNo, ParkingFacility.occupancy	Vehicle.normalizeRegNo(); ParkingFacility.getTotalOccupancy()	Filling the RATE_PER_HOUR table

3. ParkingTicket Object Life Cycle

Figure 1: ParkingTicket object life cycle



Eligible != destroyed: the JVM decides when (or whether) memory is reclaimed.

Stages

- **1. Creation:** a vehicle arrives and `ParkingService.enter()` finds a free slot through `ParkingFacility.allocate()` and executes `new ParkingTicket(...)`; the object is allocated on the heap.
- **2. Number generation:** the constructor calls `SEQUENCE.incrementAndGet()` and stores the result in its own final `ticketNo`.

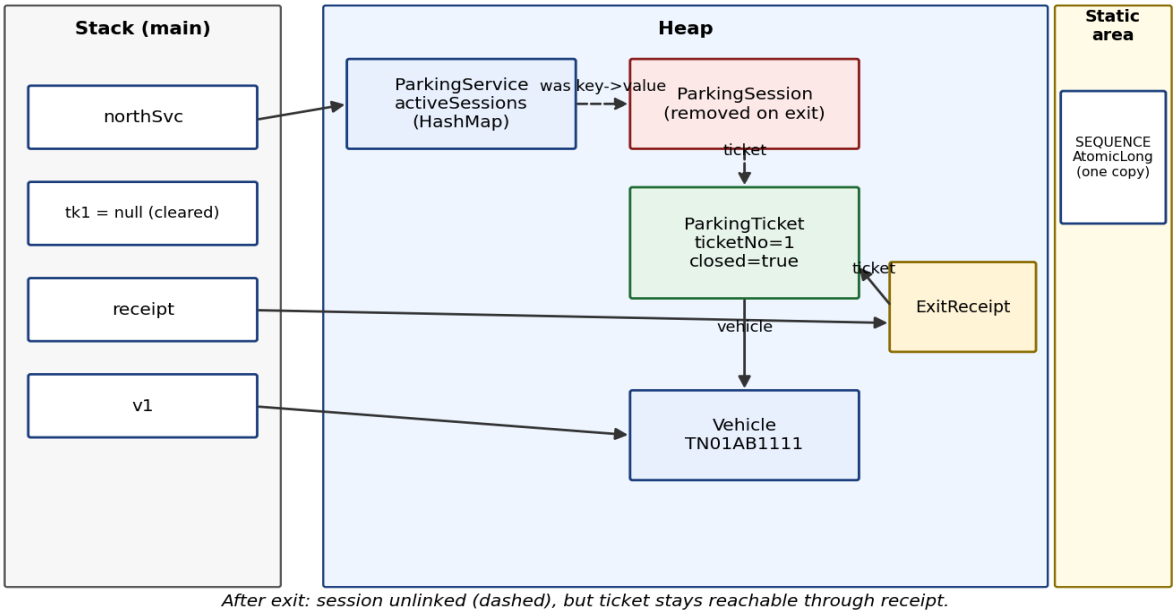
- **3. Association:** the ticket holds references to its `Vehicle`, facility name, floor, slot and entry time. A `ParkingSession` (ticket + facility) is created and put into the service's `activeSessions` map keyed by registration number.
- **4. Active period:** the ticket is strongly reachable through the session map and through the variable that received the return value of `enter()`.
- **5. Exit and closure:** `exit()` removes the session from the map, calls `facility.release(floor, slot)` (which lowers that facility's occupancy and the static total occupancy), calls `ticket.close()`, computes the fee and returns an `ExitReceipt`.
- **6. Session removal:** `activeSessions.remove(regNo)` removes the map's reference to the session; the session then becomes unreachable from the service (its ticket reference is no longer reachable through it).
- **7. Garbage-collection eligibility:** see below.

Garbage collection explained accurately

Java does **not** destroy a ticket at the moment the vehicle leaves. Closing a ticket only changes its `closed` flag. An object becomes *eligible* for garbage collection only when **no strong reference chain from a GC root** (local variables on active stacks, static fields, live threads) can reach it. Removing one reference is therefore not enough: the ticket stays reachable while any other reference exists, for example the local variable in the caller that received it from `enter()`, a list kept for reporting, or an `ExitReceipt`.

In this design `ExitReceipt` keeps a `ParkingTicket` reference. As long as the receipt is reachable, its ticket is reachable too, even though the session has been removed. This is shown in Figure 2 and in the demo, where after `tk1 = null` the ticket can still be read through `receipt.getTicket()`. Once the caller drops the receipt and every other reference, the ticket (and, if nothing else refers to them, its session and vehicle) becomes eligible. **When** the collector actually reclaims it is decided by the JVM; `System.gc()` is only a request, so timing is never guaranteed.

Figure 2: References after exit() - memory view



After exit: session unlinked (dashed), but ticket stays reachable through receipt.

4. Object Passing and Returning in Java

Java always passes arguments **by value**. For an object parameter the value copied is the *reference*, so the caller's variable and the method's parameter are two separate variables that point to the same object.

A. Passing objects to methods

```
static void updateOwner(Vehicle v) {
    v.setOwnerName("Updated Owner");                    // changes the shared object
    v = new Vehicle("TN99ZZ9999", "CAR", "Local Only");   // changes only the local copy of the reference
    System.out.println("  inside method, local v -> " + v);
}
```

- The first statement works through the copied reference and mutates the one `Vehicle` object, so the caller sees the change (output: *owner=Updated Owner*).
- The second statement reassigns only the parameter. The caller's variable `v1` still refers to the original object (output after the call still shows TN01AB1111).

B. Returning objects from methods

```
public ParkingTicket enter(Vehicle vehicle, LocalDateTime entryTime) { ... return ticket; }
public ExitReceipt   exit(String regNo, LocalDateTime exitTime)     { ... return new ExitReceipt(...); }
```

`enter()` returns a reference to the newly created ticket (or `null` if the facility is full), so the gate can print or store it. `exit()` returns an `ExitReceipt` that packages ticket, minutes and fee together, so a method can hand back several related values without static or shared variables. Returning the reference copies only the reference, not the object, so no duplicate ticket is made.

5. Static vs Instance Members – Comparison

Aspect	Static member	Instance member
Ownership	The class	One specific object
Memory association	Class-level storage, created at class initialisation	Part of each object on the heap, created with *new*
Number of copies	One per class	One per object
Access method	ClassName.member	objectReference.member
Lifetime	From class initialisation until the class is unloaded (normally program end)	From object creation until the object is unreachable and collected
Parking examples	Global ticket sequence (ParkingTicket.SEQUENCE); total occupancy across all facilities (ParkingFacility.totalOccupancy); rate table	Occupancy count of each ParkingFacility; vehicle registration number; ticketNo of each ticket

Data item	Kind	Reason
Global ticket sequence	static	Common to all tickets and both facilities
Total occupancy (all facilities)	static (ParkingFacility.totalOccupancy)	One system-wide value shared by every facility; read with static getTotalOccupancy()
Occupancy count of one facility	instance (ParkingFacility.occupancy)	North and South must count independently; read with getOccupancy()
Vehicle registration number	instance (Vehicle)	Different for every vehicle
Ticket number	instance field, generated from static sequence	Each ticket must keep its own number

6. What If the Ticket Counter Were an Instance Variable?

Case 1 – Static ticket counter

Both facilities share one counter. Ticket numbers stay globally unique within the running application, and creating more `ParkingFacility` objects does not create more counters.

```
static class StaticCounterFacility {
    private static final AtomicLong COUNTER = new AtomicLong(0);
    private final String name;
    String issue() { return name + "-ticket#" + COUNTER.incrementAndGet(); }
}
// North.issue(), South.issue(), North.issue()  ->  North-ticket#1, South-ticket#2, North-ticket#3
```

Case 2 – Instance ticket counter

Each facility has its own counter, so North Gate issues ticket 1 and South Gate independently also issues ticket 1. Compared globally, two different tickets now carry the same identifier, breaking the “ticket number is never reused” rule.

```
static class InstanceCounterFacility {
    private long counter = 0;                       // one copy per facility
    String issue() { return name + "-ticket#" + (++counter); }
}
// North.issue(), South.issue(), North.issue()  ->  North-ticket#1, South-ticket#1, North-ticket#2
```

Case 3 – Recommended solution

Use **one shared static sequence** for globally unique numbers. If the university prefers separate numbering per gate, keep an instance counter but make the identifier a combination of gate code and local number (e.g. N-1, S-1, N-2); the pair is unique even though the numbers repeat. The demo prints *Gate-prefixed: N-1, S-1, N-2*. **Limitation:** an in-memory static counter restarts from zero when the application restarts, so it guarantees uniqueness only within one run. Uniqueness across restarts requires durable storage (for example a database sequence or a persisted last-used value).

7. Space Complexity Analysis

Let **k** = number of ParkingFacility objects and **n** = number of ParkingTicket objects.

Design	Counter storage	Ticket object storage
Static counter	O(1)	O(n)
Instance counter	O(k)	O(n)

- One static counter is shared by all k facilities, so counter storage is constant, O(1).
- An instance counter needs one counter per facility, so counter storage is O(k).
- Both designs still need O(n) space for n tickets, because each ticket object stores its own data (ticketNo, vehicle reference, times, and so on).
- Total ticket-counter-plus-ticket space is O(1 + n) = O(n) for static and O(k + n) for instance. The static design saves only the k–1 extra ticket counters; the O(1) figure applies **to the ticket counter only**, not to the whole parking system.
- **Occupancy counters are a separate matter:** the per-facility `occupancy` fields need one int per facility, O(k) across k facilities, and the single static `totalOccupancy` needs O(1). Together they use O(k + 1) = O(k) space; this is independent of the ticket-counter choice and is not merged with the ticket-counter figures above.

8. Advantages and Limitations

Criterion	Static counter	Instance counter (per facility)
Global uniqueness	Guaranteed within one run	Not guaranteed; duplicates across facilities (fixed only with a gate prefix)
Independent facility numbering	Not available; numbers interleave	Available
Memory usage	O(1) for the counter	O(k) for counters
Maintainability	Single place to change the numbering rule	Rule repeated per object; prefix scheme needs extra code
Scalability	Adding facilities needs no change; AtomicLong is thread-safe; a single counter may become a contention point only at very high rates, and persistence needs external storage	Facilities scale independently, but global identification needs the gate id in every ticket reference
Suitability for this system	High: one university, one ticket sequence	Only acceptable with a gate prefix

Final design decision: keep `private static final AtomicLong SEQUENCE` in `ParkingTicket` for globally unique ticket numbers. In `ParkingFacility` keep `private int occupancy` as an **instance** variable so each facility reports its own count, and add `private static int totalOccupancy` for the system-wide count. This satisfies the unique-ticket rule for both gates with minimal memory, gives independent per-facility occupancy and a correct total, and both occupancy counters change only on a successful allocation or release. If numbering must survive restarts, the sequence should be backed by durable storage.

9. Java Source Code

All files are in the default package, as in Tasks 1–4, and compile together with `javac *.java`.

Vehicle.java

```
import java.util.Map;

/** Vehicle used by the parking system (kept minimal; hierarchy is Task 6 scope). */
public class Vehicle {
    // Static constant table, filled once when the class is initialised (static block)
    private static final Map<String, Integer> RATE_PER_HOUR;

    static {
        System.out.println("[Vehicle static block] executed once at class initialisation");
        RATE_PER_HOUR = Map.of("TWO_WHEELER", 10, "CAR", 30, "EV", 30);
    }

    private final String regNo;      // instance-specific
    private final String type;       // instance-specific
    private String ownerName;        // mutable state, used to demonstrate parameter passing

    public Vehicle(String regNo, String type, String ownerName) {
        this.regNo = normalizeRegNo(regNo);
        this.type = type;
        this.ownerName = ownerName;
    }

    /** Static utility: needs no Vehicle object and uses no instance data. */
    public static String normalizeRegNo(String raw) {
        if (raw == null || raw.isBlank()) {
            throw new IllegalArgumentException("Registration number is required");
        }
        return raw.replaceAll("[\\s-]", "").toUpperCase();
    }

    /** Static utility: independent fee calculation, every started hour is charged. */
    public static long billableHours(long minutes) {
        return (minutes + 59) / 60;   // integer ceiling, no floating-point casts
```

```
    }

    public static int rateFor(String type) {
        Integer r = RATE_PER_HOUR.get(type);
        if (r == null) throw new IllegalArgumentException("Unknown vehicle type: " + type);
        return r;
    }

    public String getRegNo() { return regNo; }
    public String getType() { return type; }
    public String getOwnerName() { return ownerName; }
    public void setOwnerName(String ownerName) { this.ownerName = ownerName; }

    @Override public String toString() { return type + "[" + regNo + ", owner=" + ownerName + "]"; }
}
```

ParkingTicket.java

```
import java.time.LocalDateTime;
import java.util.concurrent.atomic.AtomicLong;

public class ParkingTicket {
    /** One counter for the whole class, shared by every facility. Direct initialisation is enough. */
    private static final AtomicLong SEQUENCE = new AtomicLong(0);

    private final long ticketNo;             // each ticket keeps its own copy of its number
    private final Vehicle vehicle;
    private final String facilityName;
    private final int floor;
    private final int slot;
    private final LocalDateTime entryTime;
    private boolean closed;

    public ParkingTicket(Vehicle vehicle, String facilityName, int floor, int slot, LocalDateTime entryTime) {
        this.ticketNo = SEQUENCE.incrementAndGet();   // atomic read-modify-write
        this.vehicle = vehicle;
        this.facilityName = facilityName;
        this.floor = floor;
        this.slot = slot;
        this.entryTime = entryTime;
    }

    public long getTicketNo() { return ticketNo; }
    public Vehicle getVehicle() { return vehicle; }
    public String getFacilityName() { return facilityName; }
    public int getFloor() { return floor; }
    public int getSlot() { return slot; }
    public LocalDateTime getEntryTime() { return entryTime; }
    public boolean isClosed() { return closed; }
    public void close() { closed = true; }

    @Override public String toString() {
        return "Ticket#" + ticketNo + "(" + vehicle.getRegNo() + ", " + facilityName
                + " F" + floor + "-S" + slot + (closed ? ", CLOSED" : ", ACTIVE") + ")";
    }
}
```

ParkingSession.java

```
public class ParkingSession {
    private final ParkingTicket ticket;
    private final ParkingFacility facility;

    public ParkingSession(ParkingTicket ticket, ParkingFacility facility) {
        this.ticket = ticket;
        this.facility = facility;
    }

    public ParkingTicket getTicket() { return ticket; }
    public ParkingFacility getFacility() { return facility; }
}
```

ExitReceipt.java

```
import java.time.LocalDateTime;

public class ExitReceipt {
    private final ParkingTicket ticket;      // keeps the ticket reachable while the receipt is reachable
    private final LocalDateTime exitTime;
    private final long minutes;
    private final long fee;

    public ExitReceipt(ParkingTicket ticket, LocalDateTime exitTime, long minutes, long fee) {
        this.ticket = ticket;
        this.exitTime = exitTime;
        this.minutes = minutes;
        this.fee = fee;
```

```
    }

    public ParkingTicket getTicket() { return ticket; }
    public LocalDateTime getExitTime() { return exitTime; }
    public long getMinutes() { return minutes; }
    public long getFee() { return fee; }

    @Override public String toString() {
        return "Receipt{ticket=" + ticket.getTicketNo() + ", minutes=" + minutes + ", fee=Rs " + fee + "}";
    }
}
```

ParkingFacility.java

```
public class ParkingFacility {
    private final String name;
    private final String[][] slots;      // slots[floor][slot] = reg no, or null if free
    private int occupancy;               // instance-specific: vehicles parked in THIS facility
    private static int totalOccupancy;   // static: vehicles parked across ALL facilities

    public ParkingFacility(String name, int[] slotsPerFloor) {
        this.name = name;
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    /** Returns {floor, slot} of the first free slot, or null if the facility is full. */
    public int[] allocate(String vehicleNo) {
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    occupancy++;                // successful allocation only
                    totalOccupancy++;
                    return new int[]{f, s};
                }
            }
        }
        return null;
    }

    /** Frees a slot. Returns false if the position is invalid or already free. */
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length || slot < 0 || slot >= slots[floor].length
                || slots[floor][slot] == null) {
            return false;
        }
        slots[floor][slot] = null;
        occupancy--;                    // successful release only
        totalOccupancy--;
        return true;
    }

    public String getName() { return name; }
    public int getOccupancy() { return occupancy; }                     // this facility
    public static int getTotalOccupancy() { return totalOccupancy; }    // all facilities
}
```

ParkingService.java

```
import java.time.Duration;
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

public class ParkingService {
    private final ParkingFacility facility;
    private final Map<String, ParkingSession> activeSessions = new HashMap<>();

    public ParkingService(ParkingFacility facility) { this.facility = facility; }

    /** Receives a Vehicle reference, returns a ParkingTicket reference (or null if full). */
    public ParkingTicket enter(Vehicle vehicle, LocalDateTime entryTime) {
        if (activeSessions.containsKey(vehicle.getRegNo())) {
            throw new IllegalStateException("Vehicle already parked: " + vehicle.getRegNo());
        }
        int[] pos = facility.allocate(vehicle.getRegNo());
        if (pos == null) return null;
        ParkingTicket ticket = new ParkingTicket(vehicle, facility.getName(), pos[0], pos[1], entryTime);
        activeSessions.put(vehicle.getRegNo(), new ParkingSession(ticket, facility));
        return ticket;
    }

    /** Closes the ticket, frees the slot, removes the session and returns an ExitReceipt. */
    public ExitReceipt exit(String regNo, LocalDateTime exitTime) {
```

```
        ParkingSession session = activeSessions.remove(Vehicle.normalizeRegNo(regNo)); // removes one reference
        if (session == null) throw new IllegalStateException("No active session for " + regNo);
        ParkingTicket ticket = session.getTicket();
        facility.release(ticket.getFloor(), ticket.getSlot());
        ticket.close();
        long minutes = Duration.between(ticket.getEntryTime(), exitTime).toMinutes();
        long fee = Vehicle.billableHours(minutes) * Vehicle.rateFor(ticket.getVehicle().getType());
        return new ExitReceipt(ticket, exitTime, minutes, fee);
    }

    public int activeCount() { return activeSessions.size(); }
    public ParkingFacility getFacility() { return facility; }
}
```

Task5Demo.java

```
import java.lang.ref.WeakReference;
import java.time.LocalDateTime;
import java.util.concurrent.atomic.AtomicLong;

public class Task5Demo {

    /** Demo-only class: the WRONG design, with an instance ticket counter. */
    static class InstanceCounterFacility {
        private final String name;
        private long counter = 0;               // one copy per facility object
        InstanceCounterFacility(String name) { this.name = name; }
        String issue() { return name + "-ticket#" + (++counter); }
    }

    /** Demo-only class: static counter shared by all objects. */
    static class StaticCounterFacility {
        private static final AtomicLong COUNTER = new AtomicLong(0);
        private final String name;
        StaticCounterFacility(String name) { this.name = name; }
        String issue() { return name + "-ticket#" + COUNTER.incrementAndGet(); }
    }

    /** Demo-only: facility id + local sequence (alternative scheme). */
    static class PrefixedFacility {
        private final String gate;
        private long local = 0;
        PrefixedFacility(String gate) { this.gate = gate; }
        String issue() { return gate + "-" + (++local); }
    }

    // ---- Object passing: the reference VALUE is copied ----
    static void updateOwner(Vehicle v) {
        v.setOwnerName("Updated Owner");                      // mutates the shared object
        v = new Vehicle("TN99ZZ9999", "CAR", "Local Only");    // only the local copy of the reference changes
        System.out.println("  inside method, local v -> " + v);
    }

    static void printTicket(ParkingTicket t) {
        System.out.println("  received " + t);
    }

    public static void main(String[] args) throws Exception {
        System.out.println("=== 1. Static utility method ===");
        System.out.println(Vehicle.normalizeRegNo(" tn-01 ab 1111 "));
        System.out.println("billableHours(130) = " + Vehicle.billableHours(130));

        System.out.println("\n=== 2. Instance-specific occupancy (North vs South) ===");
        ParkingFacility north = new ParkingFacility("North Gate", new int[]{2, 1});
        ParkingFacility south = new ParkingFacility("South Gate", new int[]{2, 1});
        ParkingService northSvc = new ParkingService(north);
        ParkingService southSvc = new ParkingService(south);

        LocalDateTime t0 = LocalDateTime.of(2026, 11, 2, 9, 0);
        Vehicle v1 = new Vehicle("TN01AB1111", "CAR", "Asha");
        Vehicle v2 = new Vehicle("TN02CD2222", "TWO_WHEELER", "Ravi");
        Vehicle v3 = new Vehicle("TN03EF3333", "EV", "Meena");

        System.out.println("=== 3. Returning ParkingTicket from enter() ===");
        ParkingTicket tk1 = northSvc.enter(v1, t0);
        ParkingTicket tk2 = northSvc.enter(v2, t0.plusMinutes(5));
        ParkingTicket tk3 = southSvc.enter(v3, t0.plusMinutes(10));
        System.out.println(tk1);
        System.out.println(tk2);
        System.out.println(tk3);
        System.out.println("North occupancy = " + north.getOccupancy()
                + ", South occupancy = " + south.getOccupancy()
                + ", Total occupancy = " + ParkingFacility.getTotalOccupancy());

        System.out.println("\n=== 4. Passing objects to methods ===");
        printTicket(tk1);
        System.out.println("  before: " + v1);
```

```
        updateOwner(v1);
        System.out.println("  after : " + v1 + "  (state changed, caller's variable still refers to the same Vehicle)");

        System.out.println("\n=== 5. Returning ExitReceipt from exit() and life cycle ===");
        ExitReceipt receipt = northSvc.exit("tn01ab1111", t0.plusMinutes(130));
        System.out.println(receipt);
        System.out.println("North occupancy = " + north.getOccupancy()
                + ", South occupancy = " + south.getOccupancy()
                + ", Total occupancy = " + ParkingFacility.getTotalOccupancy());
        System.out.println("Active sessions in north service = " + northSvc.activeCount());
        System.out.println("tk1 closed? " + tk1.isClosed());

        // tk1 and receipt still hold references, so tk1 remains reachable.
        WeakReference<ParkingTicket> weak = new WeakReference<>(tk1);
        tk1 = null;                                  // drop the local variable
        System.gc();                                 // only a request; not guaranteed
        System.out.println("tk1 variable cleared, receipt still references ticket -> weak.get() != null: "
                + (weak.get() != null) + " (reachable via ExitReceipt)");
        System.out.println("Ticket via receipt: " + receipt.getTicket());

        System.out.println("\n=== 6. Static vs instance counter, North Gate and South Gate ===");
        StaticCounterFacility sn = new StaticCounterFacility("North");
        StaticCounterFacility ss = new StaticCounterFacility("South");
        System.out.println("Static : " + sn.issue() + ", " + ss.issue() + ", " + sn.issue());

        InstanceCounterFacility in = new InstanceCounterFacility("North");
        InstanceCounterFacility is = new InstanceCounterFacility("South");
        String a = in.issue(), b = is.issue();
        System.out.println("Instance: " + a + ", " + b + ", " + in.issue());
        System.out.println("Instance duplicate number across facilities: "
                + a.substring(a.indexOf('#')).equals(b.substring(b.indexOf('#'))));

        PrefixedFacility pn = new PrefixedFacility("N"), ps = new PrefixedFacility("S");
        System.out.println("Gate-prefixed: " + pn.issue() + ", " + ps.issue() + ", " + pn.issue());

        System.out.println("\n=== 7. Invalid release and failed allocation leave counters unchanged ===");
        int totalBefore = ParkingFacility.getTotalOccupancy();
        boolean r1 = north.release(5, 0);      // invalid floor
        boolean r2 = north.release(0, 0);      // slot already free (its vehicle exited)
        ParkingFacility tiny = new ParkingFacility("Tiny Lot", new int[]{0});
        int[] none = tiny.allocate("TN00XX0000");   // no free slot -> null
        System.out.println("release(5,0)=" + r1 + ", release(0,0)=" + r2 + ", allocate on full lot="
                + java.util.Arrays.toString(none));
        System.out.println("North occupancy = " + north.getOccupancy()
                + ", South occupancy = " + south.getOccupancy()
                + ", Tiny occupancy = " + tiny.getOccupancy()
                + ", Total occupancy = " + ParkingFacility.getTotalOccupancy()
                + " (before: " + totalBefore + ")");
    }
}
```

10. Code Explanation

- **AtomicLong ticket numbers:** `ParkingTicket`'s constructor calls `SEQUENCE.incrementAndGet()`, which atomically adds 1 and returns the new value, and stores it in `ticketNo`. The output shows Ticket#1, #2 and #3 across both facilities.
- **Why static:** `SEQUENCE` is declared `static final`, so only one counter exists for all ParkingTicket objects and both gates; `final` prevents the reference from being replaced.
- **Why occupancy is an instance variable and total occupancy is static:** `ParkingFacility.occupancy` is incremented in `allocate()` and decremented in `release()` of that object only, so North and South stay independent (1 and 1 after the exit). `ParkingFacility.totalOccupancy` is static, is changed by the same two methods in every facility object, and is read with `getTotalOccupancy()` (3 after the three entries, 2 after the exit). Both are changed only after a successful allocation or release; section 7 of the demo shows an invalid release and an allocation on a full lot leaving the total at 2.
- **Static methods:** `Vehicle.normalizeRegNo`, `billableHours` and `rateFor` are called as `Vehicle.method(...)`. The constructor itself calls `normalizeRegNo`, so all registration numbers are stored normalised.

- **Static block:** the block in Vehicle runs once when Vehicle is first initialised; the demo output prints its message once, before the first normalisation result.
- **Passing/returning references:** `updateOwner(Vehicle)` shows mutation versus reassignment; `enter()` returns the ticket and `exit()` returns the receipt.
- **Garbage-collection eligibility:** in `main`, `tk1` is set to null but `receipt` still refers to the ticket, so the WeakReference still returns the object (printed true). Eligibility would begin only after the receipt and all other references are gone; the program does not (and cannot) force that moment.

11. Result

Appropriate static members (ticket sequence, total occupancy across all facilities, rate table, utility methods) and instance members (per-facility occupancy, registration number, ticket number) were identified. Static variables, static methods and a static initialization block were demonstrated, with North Gate occupancy 2, South Gate occupancy 1 and total occupancy 3, falling to 1, 1 and 2 after one North Gate exit. The ParkingTicket life cycle and the conditions for garbage-collection eligibility were explained. Object passing and returning were demonstrated with `updateOwner`, `enter()` and `exit()`. The static, instance and gate-prefixed ticket-counter approaches were compared, and memory was analysed as O(1) versus O(k) for ticket counters, O(k) for per-facility occupancy counters with O(1) for the static total, and O(n) for n tickets.

Execution status: the seven source files compiled without warnings (`javac -Xlint:all`) and `Task5Demo` was executed on OpenJDK 21.0.12 during preparation; the output below was captured from that run. The student should re-run it on the Java version installed on their machine (the README gives the commands) and add their own screenshot as output.png.

Captured program output

```
=== 1. Static utility method ===
[Vehicle static block] executed once at class initialisation
TN01AB1111
billableHours(130) = 3

=== 2. Instance-specific occupancy (North vs South) ===
=== 3. Returning ParkingTicket from enter() ===
Ticket#1(TN01AB1111, North Gate F0-S0, ACTIVE)
Ticket#2(TN02CD2222, North Gate F0-S1, ACTIVE)
Ticket#3(TN03EF3333, South Gate F0-S0, ACTIVE)
North occupancy = 2, South occupancy = 1, Total occupancy = 3

=== 4. Passing objects to methods ===
  received Ticket#1(TN01AB1111, North Gate F0-S0, ACTIVE)
  before: CAR[TN01AB1111, owner=Asha]
  inside method, local v -> CAR[TN99ZZ9999, owner=Local Only]
  after : CAR[TN01AB1111, owner=Updated Owner]  (state changed, caller's variable still refers to the same Vehicle)

=== 5. Returning ExitReceipt from exit() and life cycle ===
Receipt{ticket=1, minutes=130, fee=Rs 90}
North occupancy = 1, South occupancy = 1, Total occupancy = 2
Active sessions in north service = 1
tk1 closed? true
tk1 variable cleared, receipt still references ticket -> weak.get() != null: true (reachable via ExitReceipt)
Ticket via receipt: Ticket#1(TN01AB1111, North Gate F0-S0, CLOSED)

=== 6. Static vs instance counter, North Gate and South Gate ===
Static : North-ticket#1, South-ticket#2, North-ticket#3
Instance: North-ticket#1, South-ticket#1, North-ticket#2
Instance duplicate number across facilities: true
Gate-prefixed: N-1, S-1, N-2

=== 7. Invalid release and failed allocation leave counters unchanged ===
release(5,0)=false, release(0,0)=false, allocate on full lot=null
North occupancy = 1, South occupancy = 1, Tiny occupancy = 0, Total occupancy = 2 (before: 2)
```